

ProtoPNet Crop Disease Research Workflow

Option B: Transfer Learning Framework for Interpretable Crop Disease Classification
PlantWild (Primary) → Crop Pest & Disease (Transfer) | XAI with ProtoPNet

PIPELINE OVERVIEW

The full research pipeline is divided into seven sequential stages. Each stage feeds directly into the next, ensuring reproducibility and a clean experimental narrative for publication.

Stage	Name	Key Output
1	Dataset Acquisition	Raw images from PlantWild + Crop Pest
2	Preprocessing	Cleaned, normalised, split image sets
3	Phase 1–4 Training on PlantWild	Saved backbone + prototypes (Model A)
4	Transfer to Crop Pest	Fine-tuned Model B with frozen backbone
5	Evaluation	Accuracy, F1, AUC on both datasets
6	XAI Analysis	Prototype visualisations + similarity maps
7	Ablation Study	Comparison vs. scratch training (Option A)

STAGE 1 — DATASET ACQUISITION

1.1 Primary Dataset: PlantWild

PlantWild is the largest in-the-wild plant disease image dataset, designed for real-world agricultural conditions rather than controlled laboratory settings.

Property	Detail
Total Images	18,542
Disease Classes	89 (56 diseased + 33 healthy)
Image Type	In-the-wild (varied background, lighting, viewpoint)

Annotation	Class labels + text descriptions per disease
Source	Kaggle: mariaboukhelf/plantwild HuggingFace: uqtwei2/PlantWild
Why Use It	No published ProtoPNet results — any result is a new SOTA for interpretable models

1.2 Secondary Dataset: Crop Pest & Disease Detection

Property	Detail
Raw Images	24,881 (Cashew: 6,549 Cassava: 7,508 Maize: 5,389 Tomato: 5,435)
Augmented	102,976 images after augmentation
Crop Classes	4 crops, ~22 disease/pest classes
Source	Kaggle: nimalsankalana/crop-pest-and-disease-detection Mendeley Data
Why Use It	Direct CPNet/ProtoPNet baselines already published — clear target to beat

Research Tip: Download both datasets before any preprocessing. Organise into /data/plantwild/ and /data/croppest/ with consistent subfolder structure: train/, val/, test/ per class.

STAGE 2 — PREPROCESSING

Apply identical preprocessing pipelines to BOTH datasets so that any performance difference is attributable to the model — not data handling.

2.1 Image Preprocessing Steps

Step	Operation	Value / Rationale
1	Resize	224 x 224 px — standard for DenseNet161 backbone
2	Normalise	ImageNet mean [0.485,0.456,0.406], std [0.229,0.224,0.225]
3	Remove Duplicates	Hash-based deduplication across train/val/test splits
4	Quality Filter	Remove blurry images (Laplacian variance < threshold)
5	Class Balance Check	Plot class distribution; flag severe imbalance (>10:1 ratio)

2.2 Data Augmentation (Training Set Only)

Augmentation	Range / Setting	Purpose
Random Horizontal Flip	p = 0.5	Viewpoint invariance
Random Vertical Flip	p = 0.3	Field image orientation variance
Random Rotation	±30 degrees	Leaf angle variation

Colour Jitter	brightness=0.3, contrast=0.3, saturation=0.2	Lighting condition variance (critical for PlantWild)
Random Crop	Scale 0.7–1.0	Focus on disease regions
Gaussian Blur	p = 0.2, kernel 3–5	Image quality variation

2.3 Dataset Split

Dataset	Train	Validation	Test	Strategy
PlantWild	70%	15%	15%	Stratified split — maintain class proportions
Crop Pest	70%	15%	15%	Same strategy; use augmented set for train only

STAGE 3 — MODEL ARCHITECTURE

ProtoPNet consists of four components stacked sequentially. DenseNet161 is recommended as the backbone because it is the architecture used by CPNet and CDPNet — the current state-of-the-art baselines you are targeting.

3.1 Architecture Components

Component	Description	Key Settings
Backbone (Feature Extractor)	DenseNet161 pre-trained on ImageNet. Extracts high-level convolutional feature maps from input images.	Freeze during warm-up. Output: 7x7 feature maps at 2208 channels.
Add-on Layers	Two 1x1 convolutional layers that reduce feature dimensions and prepare them for prototype comparison.	Output channels: 128 or 256. ReLU activation. Always trainable.
Prototype Layer	Core of ProtoPNet. Contains K prototypes per class (K=10 recommended). Each prototype is a learned latent patch vector. Similarity is computed as L2 distance to each spatial location in the feature map.	PlantWild: 89 classes x 10 = 890 prototypes. Crop Pest: 22 classes x 10 = 220 prototypes. Distance metric: L2 (or cosine with ProtoPNeXt).
Fully Connected Classification Layer	Single linear layer from prototype activations to class logits. Weights are constrained: positive connections only within same class, negative across classes.	No bias term. Weight constraint enforced after each update.

3.2 Loss Function

ProtoPNet uses a composite loss with three terms:

- Cross-Entropy Loss — standard classification loss
- Cluster Loss — pushes each training patch to be close to at least one prototype of its own class. Encourages prototypes to represent real disease features.
- Separation Loss — pushes training patches away from prototypes of other classes. Increases inter-class discrimination.
- L1 Regularisation on the final layer — keeps the classification layer sparse and interpretable.

For PlantWild specifically: upweight the Cluster Loss coefficient (try 0.8 vs default 0.8) because PlantWild has large intra-class variance. This forces prototypes to be more representative of the true visual disease pattern rather than background noise.

STAGE 4 — TRAINING PROCEDURE (OPTION B)

4.1 Phase-by-Phase Training on PlantWild

ProtoPNet training is divided into four distinct phases. Each phase has a different set of trainable parameters.

Phase 1 Warm-up	Trainable: Add-on layers + FC classification layer only. Backbone FROZEN. Run for 5 epochs. Goal: Initialise the add-on layers before full joint training. LR: 3e-3 for add-on layers, 3e-3 for FC layer.
Phase 2 Joint Training	Trainable: ALL layers (backbone unfrozen). Run for 20–30 epochs. Prototype push every 5 epochs. Goal: Learn discriminative feature space and push prototypes toward real patches. LR: 1e-4 backbone, 3e-3 add-on, 3e-3 FC. Use cosine annealing scheduler.
Phase 3 Prototype Push	No gradient updates — pure forward pass. For each prototype: find the training image patch with highest similarity. Replace prototype vector with that actual patch latent vector. Goal: Ensure every prototype corresponds to a real, interpretable image region. Run after every 5 epochs of Phase 2, and once finally after Phase 2 ends.
Phase 4 FC Fine-tune	Trainable: ONLY the final classification layer. Run for 20 epochs. Goal: Re-optimize classification weights after prototype positions are fixed. LR: 1e-4. Apply L1 regularisation on FC weights.

Save Model A: After Phase 4 completes on PlantWild, save the full model checkpoint. This includes backbone weights, prototype vectors, add-on layer weights, and FC layer weights.

4.2 Transfer Training on Crop Pest (Option B Core Step)

Load Model A. Freeze the backbone. Reinitialise the prototype layer and FC layer for Crop Pest classes. Then run phases 2–4 only.

Step	Action	Rationale
T1	Load Model A weights	Start from PlantWild-learned backbone
T2	Freeze backbone completely	Preserve disease feature representations from PlantWild
T3	Reinitialise prototype layer (22 classes x 10)	New prototypes for Crop Pest classes
T4	Reinitialise FC layer (22 output nodes)	New classification weights for Crop Pest
T5	Run Phase 2 (Joint — add-on + proto + FC only)	Adapt upper layers to Crop Pest images
T6	Run Phase 3 (Prototype push on Crop Pest images)	Prototypes now point to Crop Pest patches
T7	Run Phase 4 (FC fine-tune)	Final classification layer optimisation

T8	Save as Model B	This is your transfer-learned model
----	-----------------	-------------------------------------

Key Insight: The backbone in Model B still contains the feature extraction knowledge from 18,542 PlantWild images — a much larger and more diverse training set than Crop Pest alone. This is why Model B should outperform training from scratch on Crop Pest.

STAGE 5 — EVALUATION STRATEGY

5.1 Three Experiments to Run

Experiment	Train Set	Test Set	Model Used	Purpose
Exp 1	PlantWild train	PlantWild test	Model A	Primary benchmark — new SOTA for interpretable models on PlantWild
Exp 2	Crop Pest train (transfer)	Crop Pest test	Model B	Beat CPNet and vanilla ProtoPNet on established benchmark
Exp 3	PlantWild train only	Crop Pest test	Model A (no retrain)	Zero-shot cross-dataset generalisation — strongest novelty claim

5.2 Metrics to Report

Metric	Why Important
Top-1 Accuracy	Primary comparison metric — all baselines report this
Macro F1 Score	Handles class imbalance — critical for PlantWild's 89 classes
Per-class Accuracy	Identify which disease types the model struggles with
AUC-ROC (macro avg)	Measures discrimination ability beyond threshold sensitivity
Confusion Matrix	Visualise class confusions — especially similar-looking diseases
Prototype Purity	XAI metric: what % of prototypes activate on correct class regions

5.3 Ablation Study (Required for Strong Paper)

Run Option A (train from scratch on Crop Pest) alongside Option B. This ablation directly validates the value of PlantWild pre-training.

Model Variant	Description	Expected Result
Baseline ProtoPNet	Vanilla ProtoPNet from scratch on Crop Pest	Lowest accuracy — your floor
CPNet	ProtoPNet + contrastive learning from scratch	Current SOTA — your target
Your model (Option A)	Your ProtoPNet variant from scratch on Crop Pest	Should beat CPNet

Your model (Option B)	Transfer from PlantWild to Crop Pest	Should beat Option A — key result
-----------------------	--------------------------------------	-----------------------------------

STAGE 6 — XAI ANALYSIS

Explainability is ProtoPNet's primary advantage over black-box models. This stage generates the figures and analysis that distinguish your paper from standard CNN classification work.

6.1 Prototype Visualisation

Visualisation	What to Show	Where in Paper
Prototype Image Patches	For each class, show the top-3 most activated prototype patches from test images. Show side-by-side: prototype patch from training image vs. closest patch from test image.	Main Figure — shows the 'this looks like that' reasoning
Prototype Push Result	Before vs after prototype push: show that prototypes moved from abstract vectors to actual disease patches.	Supplementary or Methods figure
Class Prototype Grid	For PlantWild's 89 classes, create a grid showing one representative prototype per class. Demonstrates model learned distinct visual features per disease.	Supplementary — powerful visual for reviewers

6.2 Similarity Activation Maps

Map Type	Description	Insight
Upsampled Similarity Map	For each test image, show which spatial regions activated each prototype most strongly. Overlay as a heatmap on the original image.	Confirms model focuses on disease lesions rather than background
Top-K Activation Map	Show the top-1 prototype match for a correct and incorrect prediction side by side.	Helps explain failure modes — especially for visually similar diseases
Cross-Dataset Activation	Show prototype activations from PlantWild prototypes applied to Crop Pest images (Exp 3). Which PlantWild prototypes transfer?	Core visualisation for your cross-dataset novelty claim

6.3 Interpretability Metrics

Beyond visual inspection, report quantitative interpretability scores:

- **Prototype Purity:** % of prototype patches that belong to the correct class when evaluated by a human or bounding-box annotation. Higher = more interpretable.
- **Prototype Diversity:** Average pairwise distance between prototypes of the same class. Higher = less redundant, more informative.
- **Prototype Consistency:** Across multiple training runs, do the same image regions get selected as prototypes? Measures training stability.

STAGE 7 — PAPER STRUCTURE

Section	Content	Key Tables/Figures
Abstract	Problem + gap + method + two key results (PlantWild SOTA, Crop Pest beats CPNet)	—
Introduction	Food security motivation, black-box problem, interpretability need, contribution list	—
Related Work	PlantVillage/PlantDoc/PlantWild datasets, ProtoPNet variants (CPNet, CDPNet), transfer learning in agriculture	—
Dataset	PlantWild and Crop Pest statistics, preprocessing, split details	Table: dataset comparison
Method	ProtoPNet architecture, your modifications, transfer learning procedure, loss function	Figure: architecture diagram
Experiments	All 3 experiments + ablation. Training details, hyperparameters	Table 1: PlantWild results Table 2: Crop Pest results Table 3: Cross-dataset Table 4: Ablation
XAI Analysis	Prototype visualisations, similarity maps, interpretability metrics	Figure: prototype patches Figure: activation maps
Discussion	Why transfer helps, which diseases transfer best, failure cases, limitations	—
Conclusion	Summary, real-world deployment implications, future work	—

RECOMMENDED TIMELINE

Week	Task	Deliverable
1	Download datasets, set up environment, verify class distributions	Clean train/val/test splits for both datasets
2	Preprocessing pipeline + augmentation for PlantWild	Ready-to-train PlantWild data loaders
3–4	Phase 1–4 training on PlantWild (Model A)	Saved Model A checkpoint + PlantWild test results (Exp 1)
5	Transfer to Crop Pest — Phase 2–4 with frozen backbone (Model B)	Saved Model B checkpoint + Crop Pest results (Exp 2)
6	Option A ablation: train from scratch on Crop Pest	Baseline comparison results
6	Zero-shot Exp 3: test Model A on Crop Pest	Cross-dataset generalisation results
7	XAI analysis: prototype visualisations + activation maps	All figures for the paper
8	Write paper, compile all tables, internal review	Draft manuscript

Final Advice: Your strongest contribution is Experiment 3 (zero-shot cross-dataset). Even 55-65% accuracy on Crop Pest from a model that only saw PlantWild is a compelling result because it shows the backbone learned universal disease features — not dataset-specific patterns. Frame this as a generalisability study, not just a classification paper, and target journals such as Computers and Electronics in Agriculture, Plant Methods, or Scientific Reports.